CO3020 – Computer Organization

Assignment #1 – Chapter 1 and 2

Date: 2026/10/09 Deadline: 2026/10/23

---

1. **(8%)** Consider the following table, which tracks several performance indicators for Intel desktop processors since 2010.
   The Tech column gives the minimum feature size (t). Assume that die size remains approximately constant and transistor count scales as $\left(\frac{1}{t}\right)^2$. For each indicator, calculate the compound annual improvement rate from 2010 to 2019 and the corresponding doubling time. For the Tech column, use the estimated transistor count based on the scaling assumption above.

Desktop Processor	Year	Tech	Max. Clock Speed (GHz)	Integer IPC/ core	Cores	Max. DRAM Bandwidth (GB/s)	SP Floating Point (Gflop/s)	MiB
Westmere i7-620	2010	32	3.33	4	2	17.1	107	4
Ivy Bridge i7-3770K	2013	22	3.90	6	4	25.6	250	8
Broadwell i7-6700K	2015	14	4.20	8	4	34.1	269	8
Kaby Lake i7-7700K	2017	14	4.50	8	4	38.4	288	8
Coffee Lake i7-9700K	2019	14	4.90	8	8	42.7	627	12
Imp./year (%)								
Doubles every (Y)								

2. **(10%)** Consider two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (class A, B, C, and D). P1 with a clock rate of 2.5 GHz and CPIs of 1, 2, 3, and 3, and P2 with a clock rate of 3 GHz and CPIs of 2, 2, 2, and 2. Given a program with a dynamic instruction count of 1.0E6 instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D.
   a). (2%) Which implementation is faster? Support your answer by calculating the CPU execution time of each implementation.
   b). (4%) What is the global CPI for each implementation?
   c). (4%) Find the clock cycles required in both cases.

3. **(12%)** Compilers can have a profound impact on the performance of an application. Assume that for a program, compiler A results in a dynamic instruction count of 1.0E9 and has an execution time of 1.1s, while compiler B results in a dynamic instruction count of 1.2E9 and an execution time of 1.5s.
   a). Find the average CPI for each program given that the processor has a clock cycle time of 1 ns.
   b). Assume each program retains the CPI found in part (a). If A's and B's code run on different processors with equal execution times, find the clock-rate ratio of A's processor to B's processor.
   c). A new compiler has been developed that uses only 6.0E8 instructions and has an average CPI of 1.1. What is the speedup of using this new compiler versus using compiler A or B on the original processor?

4. **(10%)** Another pitfall is expecting to improve overall performance substantially by improving only one component. Consider a computer running a program that requires 250s, with 70s spent executing FP instructions, 85s executed L/S instructions, and 40s spent executing branch instructions. The remaining execution time is spent on INT instructions.
   a). (3%) By how much is the total time reduced if the time for FP operations is reduced by 20%?
   b). (4%) If only INT execution time can change, by how many seconds and by what percentage must it be reduced to reduce total execution time by 20%?
   c). (3%) Can reducing only branch execution time reduce total execution time by 20%? Justify your answer using the maximum possible reduction.

5. **(12%)** Assume a program requires the execution of 50 × 1.0E6 FP instructions, 110 × 1.0E6 INT instructions, 80 × 1.0E6 L/S instructions, and 16 × 1.0E6 branch instructions. The CPI for each type of instruction is 1, 1, 4, and 2, respectively. Assume that the processor has a 2 GHz clock rate.
   a). Can improving only the CPI of FP instructions make the program run twice as fast? If possible, find the required CPI; otherwise, explain why the target is unattainable.
   b). Find the new L/S CPI required to make the program run twice as fast, and the ratio of the original L/S CPI to the new value.
   c). If INT and FP CPI decrease by 40%, and L/S and branch CPI decrease by 30%, calculate the new execution time, the percentage reduction in execution time, and the overall speedup.

6. **(10%)** Provide the type, assembly language instruction, both binary and hexadecimal representation of instruction described by the following MIPS fields:
   a). op=0, rs=3, rt=2, rd=3, shamt=0, funct=34
   b). op=0x23, rs=1, rt=2, const=0x4

7. **(10%) Assume delayed branching is disabled and arithmetic does not overflow.** Consider the following MIPS loop:

```
LOOP:   slt $t2, $0, $t1
        beq $t2, $0, DONE
        addi $t1, $t1, -1
        addi $s2, $s2, 2
        j LOOP

DONE:
```

   a). (2%) Assume that the register `$t1` is initialized to the value 10. What is the value in register `$s2` assuming `$s2` is initially zero?

   b). (4%) For each of the loops above, write the equivalent C code routine. Assume that the registers `$s1, $s2, $t1`, and `$t2` are integers A, B, i, and temp, respectively.

   c). (4%) If register `$t1` initially contains an integer ($N \geq 0$), how many basic MIPS instructions are dynamically executed? Include the final loop-condition check. Labels are not instructions.

8. **(8%)** Translate the following loop into C. Assume that the C-level integer `i` is held in register `$t1, $s2` holds the C-level integer called `result`, and `$s0` holds the base address of the integer `MemArray`. MemArray contains at least 100 signed 4-byte integers and is word-aligned. Assume arithmetic does not overflow. Preserve the initial value of result; the given code does not initialize it to zero.

```
        addi $t1, $0, $0
LOOP:   lw $s1, 0($s0)
        add $s2, $s2, $s1
        addi $s0, $s0, 4
        addi $t1, $t1, 1
        slti $t2, $t1, 100
        bne $t2, $0, LOOP
```

9. **a). (10%)** Translate the following C code to MIPS assembly code. Use a minimum number of instructions. Assume that the values of `a, b, i,` and `j` are in registers `$s0, $s1, $t0`, and `$t1`, respectively. Also, assume that register `$s2` holds the base address of the array D. Keep `$s0, $s1`, and `$s2` unchanged. Assume that a and b are nonnegative, D contains signed 4-byte integers, its base address is word-aligned, and its capacity is sufficient. Arithmetic and address calculations do not overflow.

```
for(i=0; i<a; i++)
   for(j=0; j<b; j++)
      D[4*j] = i + j;
```

b). (10%) Programming Warm-up in MARS

(The detailed usage introduction of MARS is provided in "Introduction to MARS".)

Implement the 9.(a) MIPS assembly code in MARS with basic MIPS instruction. Test your program with a = 3 and b = 4. Initialize all 16 elements of D to zero. After execution, show D[0], D[4], D[8], and D[12] in the Data Segment. Run your program with Tools > Instruction Counter connected, and submit a screenshot showing the total executed instruction count and the R-, I-, and J-type counts.

Please upload

(1) `.asm` file, and

(2) a PDF file with the code explanation and the result screenshot.